

MAIE 6000C: Engineering AI Products: From Prototype to Production

Week 4 — Data Persistence and Schema Design

Program: MSc in AI & Entrepreneurship

Term: Fall 2026/27

Instructor: Lfteris NTAFL0S, PhD

The Hong Kong University of Science and Technology

Today's focus

- Persistent entities and relational data modelling
- Tables, keys, and relationships
- Schema boundaries and state transitions
- CRUD patterns and auditability
- Migrations and schema evolution
- What to persist, what to log, and what to derive
- Week 4 Team Architecture Proposal and Project Plan

Key connection:

Data design shapes the API, worker flow, AI integration, and reliability of the entire system.



Today's Learning Goals

By the end of today, we should be able to:

- Identify the core persistent entities in our system
- Design basic relational tables and relationships
- Explain primary keys, foreign keys, and status fields
- Reason about valid workflow state transitions
- Explain why migrations matter
- Distinguish persisted data, logs, and derived views
- Produce a realistic architecture and project plan

Key idea:

A weak schema creates downstream engineering problems everywhere else.



Why Data Persistence Matters

Data Persistence Is an Engineering Decision

- A system needs to remember:
- What happened
- What exists now
- What should happen next
- Who or what made a decision
- What failed, retried, or requires review

Without reliable persistence:

- Requests disappear after processing
- Workers cannot safely resume or retry
- Users cannot inspect prior results
- Teams cannot explain system behavior
- AI outputs cannot be traced or reviewed

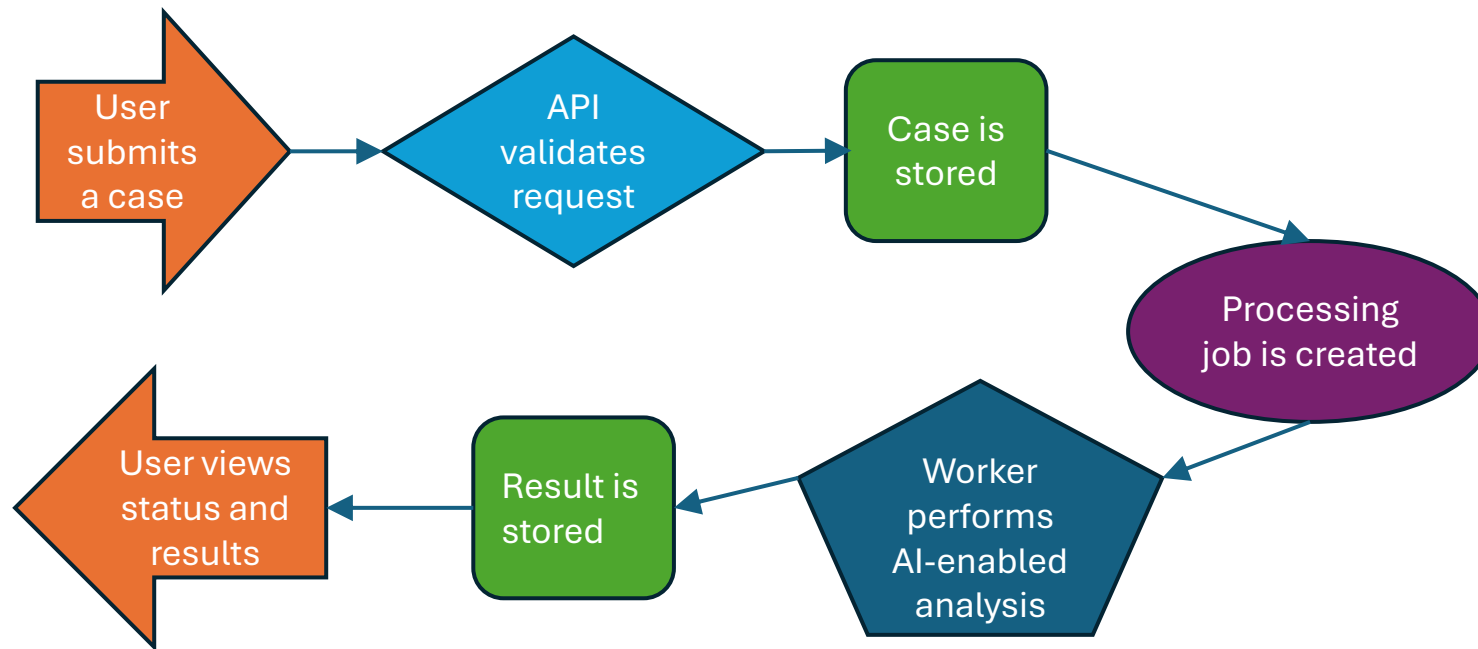
The database is often the system's source of truth.

Model the Workflow Before the Tables

Start With the Workflow, Not the Tables

Begin with one primary user action.

- Example:



Ask:

- What must survive after the request ends?
- What changes state?
- What needs a history?
- What may fail or be retried?

Identify Core Persistent Entities

Core Persistent Entities

For a typical AI-enabled workflow, possible entities include:

Entity	What it represents
users	People or system actors
cases	Main user-submitted record
jobs	Background processing work
analysis_results	AI or rules-based output
review_decisions	Human review or approval
case_events	Important workflow history

Questions to ask:

- Is this a real thing the system needs to remember?
- Does it have its own lifecycle?
- Does it need an ID?
- Does another record refer to it?

Tables, Keys, and Relationships

Example relational model (schema):

Users
id (PK)
name
email

Cases
id (PK)
submitted_by_user_id (FK → Users.id)
title
description
status
created_at

Jobs
id (PK)
case_id (FK → Cases.id)
status
attempt_count
created_at

AnalysisResults
id (PK)
job_id (FK → Jobs.id)
label
confidence
model_version

PK = Primary Key

FK = Foreign Key

Practical Schema Design Rules



A useful schema should:

- Store one fact in one appropriate place
- Use relationships instead of repeated copied data
- Give important records stable identifiers
- Record creation and update timestamps
- Use clear, validated status values
- Make important fields required when appropriate
- Support the queries your system actually needs

Avoid both extremes:

- One giant unstructured JSON field
- Too many tables for a tiny, simple workflow



State Transitions Make Workflows Explicit

Example case lifecycle:

draft → submitted → queued →
processing → completed

Possible exception paths:

- processing → failed
 - failed → queued (retry)
 - completed → needs_review
 - needs_review → approved / rejected
-
- A status field should answer:
 - What is true now?
 - What may happen next?
 - Who or what is allowed to change it?

CRUD Is More Than Four HTTP Methods

CRUD Patterns and Auditability

Operation	Typical API action	Database effect
Create	POST /cases	Insert a case record
Read	GET /cases/{id}	Retrieve current state
Update	PATCH /cases/{id}	Change allowed fields/status
Delete	DELETE /cases/{id}	Remove or archive a record

For important workflows, also consider:

case_events

- case_id
- event_type
- actor_type
- occurred_at
- Metadata

- ✓ **Current state tells us what is true.**
- ✓ **Event history helps explain how it became true.**

What to Persist, What to Log, What to Derive

Persist, Log, or Derive?

Category	Examples	Why
Persist	case details, job status, result, reviewer decision	Needed later as system truth
Log	request ID, timing, debug details, error stack trace	Helps diagnose operation
Derive	dashboard counts, rankings, cache values	Can be recomputed from stored data

For AI-enabled processing, consider persisting:

- Model or rules version
- Prompt/template version, if relevant
- Input reference or input fingerprint
- Output label and confidence
- Review decision or override reason

Migrations: How Schemas Evolve Safely



Migrations: Controlled Schema Evolution

A migration is a versioned change to the database structure:

Revision 001

Create cases table



Revision 002

Create jobs table



Revision 003

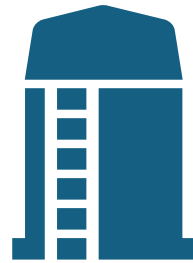
Add model_version to analysis_results

Migrations help teams:

- Keep local databases consistent
- Reproduce the schema on a new machine
- Upgrade shared environments safely
- Preserve a history of structural decisions
- Avoid “run this SQL manually” instructions

- ✓ **Changing a Python model is not enough.**
- ✓ **The actual database schema must also change.**

A Safe Migration Workflow



- Object-Relational Mapping (ORM) is a programming technique that connects app code to a database by turning database tables into classes and rows into objects.
- It links object-oriented programming languages (like Python, Java, or PHP) with relational databases (like MySQL or PostgreSQL).

Typical SQLAlchemy + Alembic flow:

```
# 1. Update the ORM model
# 2. Generate a migration revision
alembic revision --autogenerate -m "add jobs table"
# 3. Review the generated migration carefully
# 4. Apply all migrations
alembic upgrade head
```

Before committing:

- Confirm the migration matches your intended change
- Test against a clean local database
- Check whether existing data would still work
- Commit the model, migration, tests, and documentation together

Do not rewrite a shared migration that teammates may already have applied.



Data Design Mistakes to Avoid

Avoid these patterns:

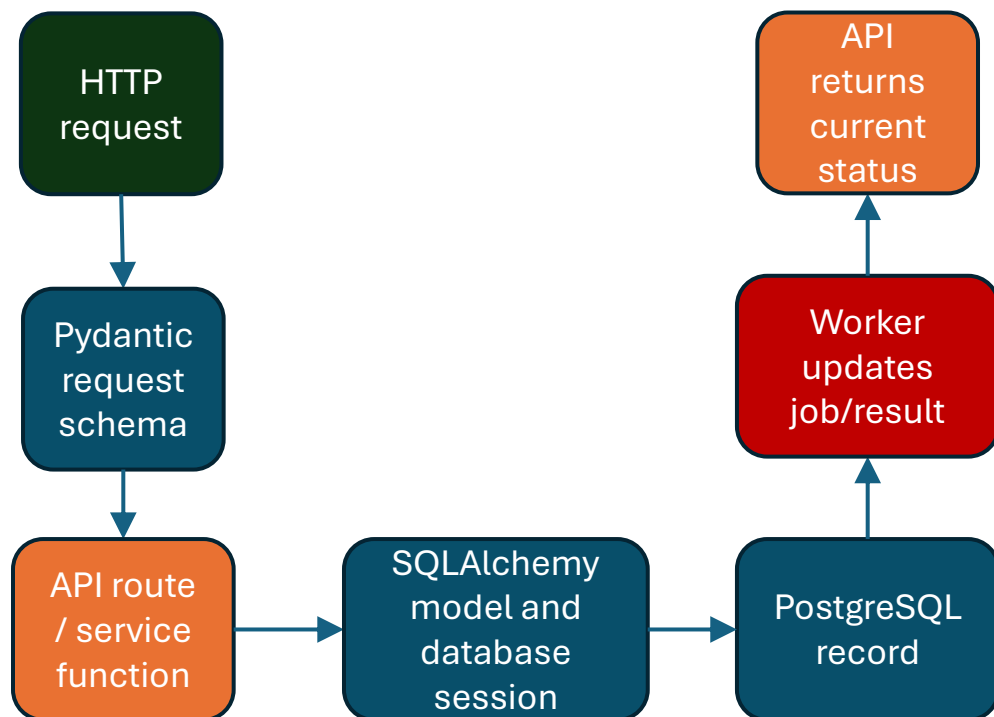
- One vague table containing everything
- One huge JSON field for all application data
- Duplicate copies of the same fact in multiple tables
- Status values with unclear meanings
- No foreign keys or relationship constraints
- Deleting records that should remain auditable
- Treating logs as the permanent source of truth
- Storing sensitive information unnecessarily
- Designing around unavailable data or paid services
- Building a schema for imagined future features

A smaller, coherent data model is stronger than a large speculative one.

Read the Persistence Layer as a System

Read the Starter Repository as a Data Flow

Trace one workflow:

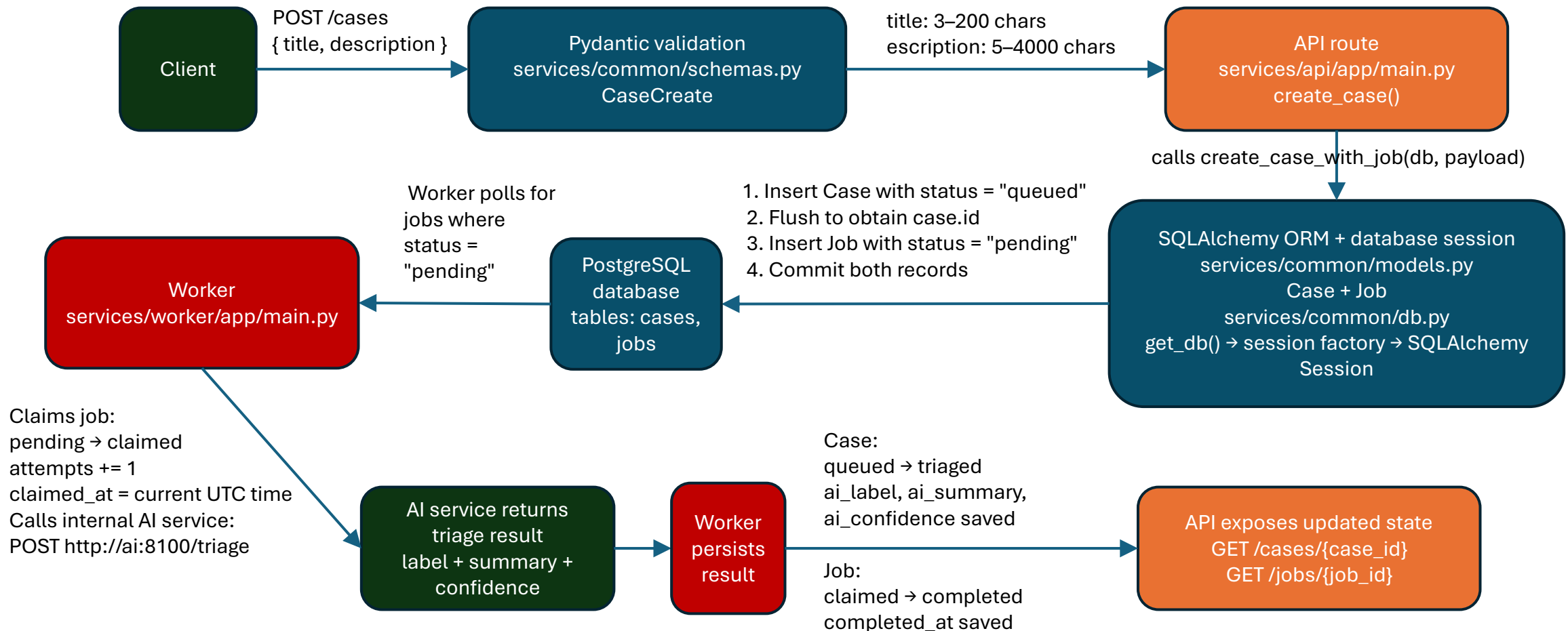


Questions to ask:

- Where are ORM models defined?
- Where are migrations stored?
- How is the database session created?
- Which records does the worker update?
- How are failures represented?
- Which tests verify persistence behavior?

Read the Persistence Layer as a System

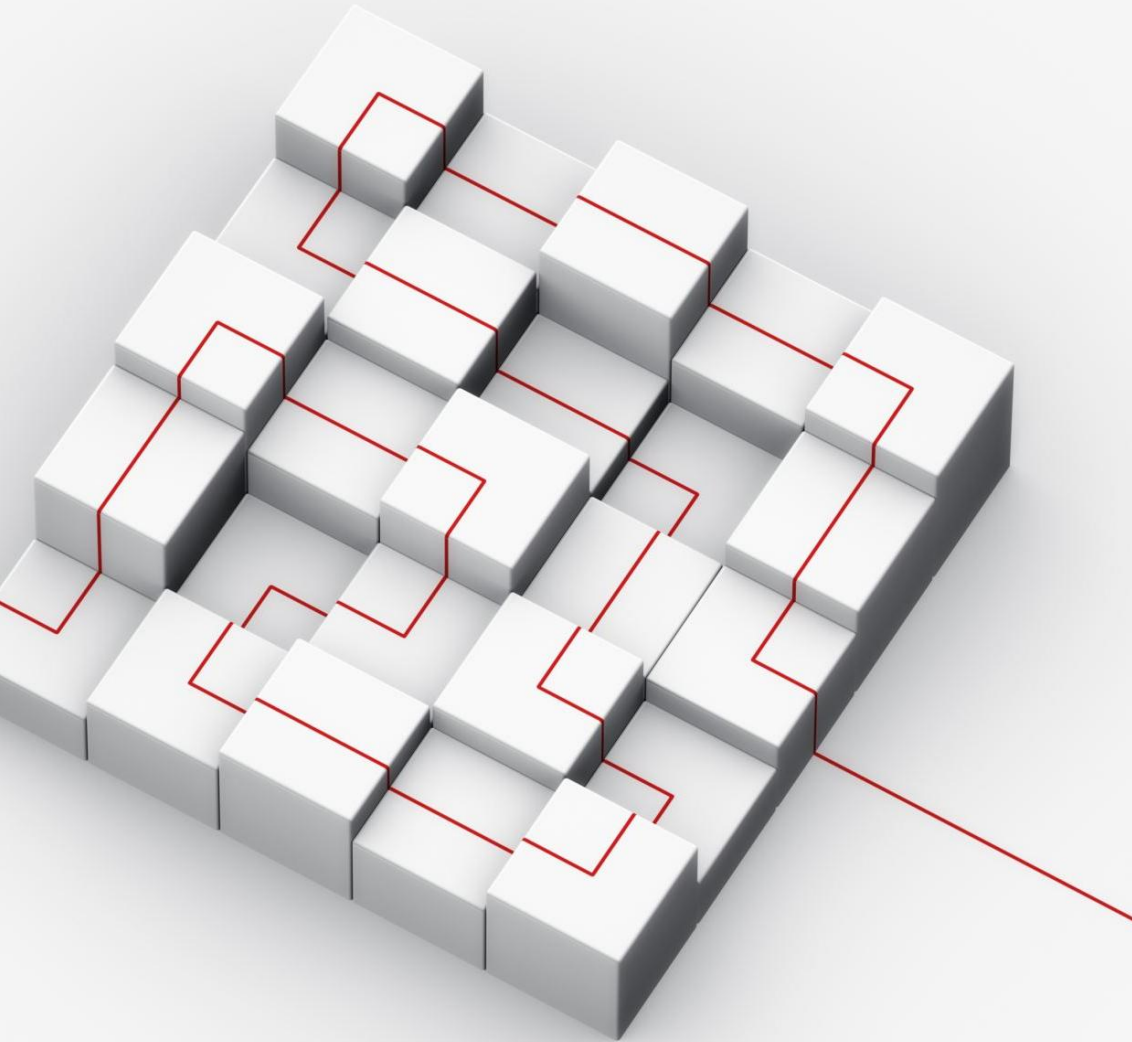
Trace One Workflow: Create and Process a Case





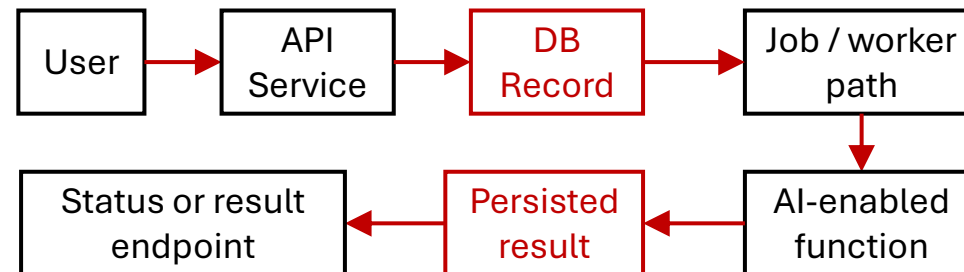
Persistence Layer: Evidence and Answers

Question	Answer / Evidence
Where are ORM models?	services/common/models.py — SQLAlchemy models: Case (cases) stores case + AI result; Job (jobs) tracks async processing. One Case can have many Jobs through Job.case_id.
Where are migrations?	alembic/versions/20260713_0001_create_initial_tables.py creates cases, jobs, and indexes. compose.yaml runs alembic upgrade head when the API starts.
How is the DB session created?	services/common/db.py: DATABASE_URL → engine → sessionmaker → get_db(). API uses Depends(get_db); worker opens sessions with get_session_factory().
What does the worker update?	Claim: Job becomes claimed, increments attempts. Success: Case becomes triaged and stores AI label/summary/confidence; Job becomes completed.
How are failures represented?	Worker marks both records as failed: Job.status = failed, saves error text and completion time; Case.status = failed. Errors are also logged.
Which tests verify it?	tests/integration/test_api_case_flow.py checks creation and retrieval of linked Case + Job. tests/smoke/test_smoke_cases.py checks the full path until the Case becomes triaged



Architecture Is a Connected Data Story

A coherent proposal should connect:



For each component, explain:

- Its responsibility
- What data it reads or writes
- What triggers it
- What happens if it fails
- How it stays within project scope

Week 4 Studio: Build the First Architecture Draft

Team Studio Output

Work with your team to produce a first draft of:

- **Primary workflow**
One user action from start to finish
- **Persistent entities**
Usually 4–7 meaningful records
- **ERD or schema diagram**
Tables, primary keys, foreign keys, relationships
- **State transitions**
Core success path and one failure/retry path
- **Architecture flow**
API → database → worker → AI → result
- **Scope and risk note**
What will you cut if time becomes limited?

W04 Team Architecture Proposal and Project Plan

Assignment type: Team

Focus: Architecture and scope planning

Required tag: w04-proposal

Your proposal should explain:

- Problem being solved
- Primary user workflow
- Proposed architecture
- Database and persistence plan
- API/service boundary plan
- Async or worker plan
- Where the AI-enabled function fits
- Milestones, risks, and scope-control plan

Required repository artifacts:

```
README.md  
docs/architecture.md  
submissions/week04/README.md
```


Submission, Tag, and Canvas Record

W04 Submission Checklist

Before submitting, confirm:

- ✓ Problem and workflow are clear
- ✓ Architecture is coherent
- ✓ Persistent entities are identified
- ✓ Async/worker path is identified
- ✓ AI-enabled function is placed in the workflow
- ✓ Milestones and likely scope cuts are realistic
- ✓ docs/architecture.md is updated
- ✓ submissions/week04/README.md exists
- ✓ Tag w04-proposal was created and pushed
- ✓ Canvas record is complete
- ✓ AI Use Statement is included if applicable



```
git tag -a w04-proposal -m "Week 4 proposal checkpoint"  
git push origin w04-proposal
```

Closing: The Data Design Habit

The Engineering Habit We Are Building

Clear workflow

- + explicit data model
- + controlled schema changes
- + traceable system state
- + realistic scope
- = reliable engineering progress

Remember:

- A smaller, coherent, persistent workflow is stronger than a large system with vague data, unclear states, and no credible plan.



Activity: Submit Your Group

- Email your group information to the TA now.
- Include the full names of all group members.
- Groups must have 2–4 students.
- Use the subject line: MAIE 6000C – Group Submission